WEB ANIMATION SCRIPTING

WAS262_SF1

25301777 Jaywin Waterboer

EcoDash: African Digital Logistics & Infrastructure Simulator

GitHub repository setup

Repository name: EcoDash-African-Logistics

Repository URL: [JPWater-cloud/EcoDash-African-Logistics: EcoDash is an African logistics simulation developed using HTML5 Canvas, CSS3 and Vanilla JavaScript for WAS262.]

For this project, I created a GitHub repository called EcoDash-African-Logistics to manage and track my work throughout the development process. The project is organised into separate files for structure, styling, and functionality. The index.html file contains the page layout, styles.css handles the visual design, and app.js contains the simulation logic.

The repository also includes a README file that explains the purpose of the project, how to run it, key features, and how AI was used during development. GitHub was used throughout the project to keep track of progress and document changes through regular commits.

1.2 African problem investigation

Problem context

EcoDash is an educational simulation that focuses on the challenge of delivering medical supplies to a rural clinic in South Africa. In many rural areas, transport infrastructure can affect how quickly goods and services reach communities. Transport accessibility plays an important role in supporting local economic activity and access to essential services (Sieber, 2009).

The simulation also introduces the concept of electric vehicle logistics by including battery management and a solar charging station. Recent research has explored how renewable energy sources could support electric delivery fleets and reduce pressure on the electricity grid (World Electric Vehicle Journal, 2025).

The scenario in EcoDash is fictional and was created to demonstrate logistics challenges in a simple and interactive way. It is not intended to represent a specific location, road network, or real delivery operation.

Physics and mathematical model

The vehicle's movement is based on simple physics calculations. The game stores the vehicle's position using x and y coordinates and tracks movement using velocity values. When the player presses the movement keys, acceleration is applied to the vehicle, causing it to move in the chosen direction.

To prevent the vehicle from moving unrealistically fast, a maximum speed limit is applied. The speed is calculated using the formula:

$$\text{speed} = \sqrt{(vx^2 + vy^2)}$$

This helps ensure that diagonal movement is balanced with horizontal and vertical movement.

The vehicle's direction is calculated using the Math.atan2() function so that the vehicle graphic rotates and faces the direction in which it is travelling.

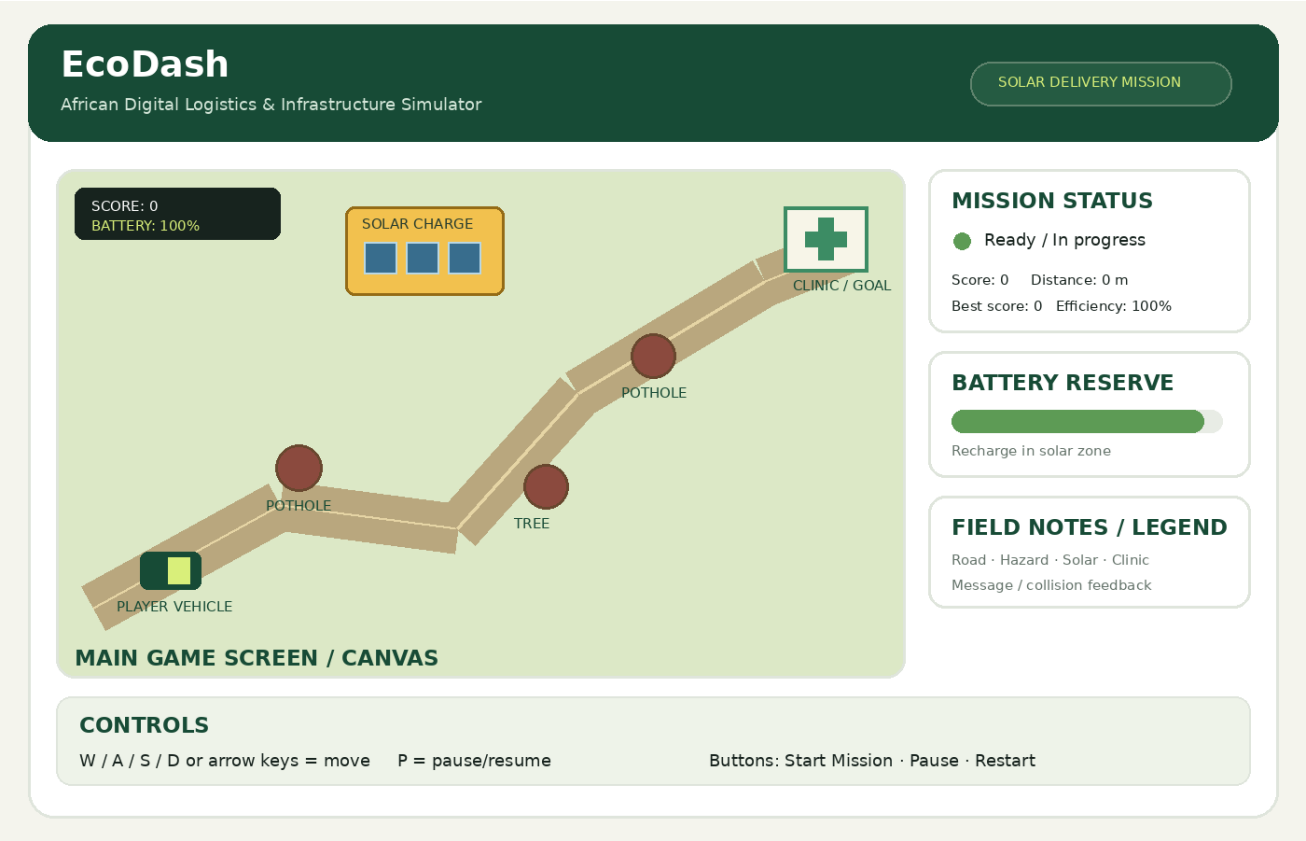
Battery power decreases while the vehicle is moving and can be restored by entering the solar charging zone. During the simulated load-shedding event, charging becomes unavailable, requiring the player to manage battery usage carefully.

Collision detection is used to identify contact with hazards, while rectangular boundary checks are used for areas such as the clinic and charging station. These mechanics are simplified for gameplay purposes and do not represent real engineering or transport data.

The game uses requestAnimationFrame() to continuously update and redraw the Canvas, creating smooth animation (MDN Web Docs, n.d.).

1.3 Wireframe

The wireframe below shows the main game screen, player, score, battery indicator, status information, field-note legend, and controls. The mini-map is optional and is not included in this design.



AI Reflection Log

Prompt used

"Help me plan a beginner-friendly HTML5 Canvas and vanilla JavaScript simulation called EcoDash. It should model an electric delivery vehicle in a South African rural environment and include movement, battery management, solar charging, obstacles, collision detection, scoring, pause/restart, and local storage. Explain the code so I can test and defend it."

Summary of AI response

AI was mainly used as a planning and learning tool during the early stages of development. It helped explain how a Canvas game could be structured using separate HTML, CSS, and JavaScript files. It also suggested approaches for movement controls, collision detection, battery management, scoring systems, game states, and browser local storage.

The explanations provided useful starting points, but the code still needed to be implemented, tested, and adjusted to suit the project requirements.

Evaluation and improvements

Although AI provided useful guidance, I could not assume that all suggestions were correct. Each feature needed to be tested in the browser to verify that it worked as intended.

During development, I checked movement mechanics, collision detection, and battery behaviour to identify any issues. The values used for speed, battery drain, charging rates, and blackout timing are simplified gameplay mechanics rather than measurements based on real-world transport systems.

Documentation was updated as the project evolved to ensure that it accurately reflected the final implementation.

One project requirement is an original feature created without generative AI assistance. This feature was developed independently and tested before being added to the final version of the simulation.

My Testing and Changes

Movement and Boundaries

I tested vehicle movement in all directions to ensure that the controls responded correctly. I also checked that the vehicle could not move outside the game area. Minor adjustments were made to the movement settings to improve responsiveness and make the controls feel smoother.

Hazard Collisions

I tested collisions by deliberately driving into hazards placed around the map. The collision detection worked as expected, and I adjusted the collision area to make interactions more consistent and fair for the player.

Battery and Solar Charging

I tested battery drain during movement and verified that charging only occurred when the vehicle entered the solar charging zone. Additional testing confirmed that charging stopped correctly during load-shedding events.

Pause, Restart, and Saved Score

The pause feature was tested during different stages of gameplay to ensure that movement and updates stopped correctly. Restart functionality was tested to confirm that game values reset properly. Local storage was also tested by refreshing the page and verifying that the saved high score remained available.

AI usage disclosure

AI use	Potential limitation	My responsibility
Project planning and structure	Suggestions may not fully match assignment requirements	Compare the work against the rubric and make necessary changes
Starter code explanations	Code may contain errors or oversimplified logic	Test, debug, and understand the code before submission
Documentation drafting	Information may be incomplete or incorrectly referenced	Verify references and update the report accurately
General programming guidance	Solutions may not fit the final implementation	Adapt suggestions based on testing and project requirements
Original feature	Must be my own independent work	Design, code, and test the feature without AI assistance

References

1. MDN Web Docs (n.d.) Canvas API. Available at: Canvas API - Web APIs | MDN (Accessed: 12 October 2026).
2. Sieber, N. (2009) Freight transport for development toolkit: rural freight. Washington, DC: World Bank. Available at: Freight transport for development toolkit : rural freight (Accessed: 12 October 2026).
3. World Electric Vehicle Journal (2025) 'Simulating the electrification of Cape Town's last-mile motorcycle fleet: Grid impact and resource requirements', World Electric Vehicle Journal, 16(12), article 641. Available at: Simulating the Electrification of Cape Town's Last-Mile Motorcycle Fleet: Grid Impact and Resource Requirements (Accessed: 12 October 2026).